

APRS-IS Consolidated Technical Reference

Architecture, client access, server processing, filtering, and IGate operation

Independent documentation compilation

2026-09-28 — complete-source draft 0.2

Contents

1	Status and scope	2
1.1	Normative language	2
2	System model	3
2.1	Network-use policy	3
2.2	RF and Internet boundaries	3
3	Line and packet representation	3
3.1	TNC2 monitor format	3
3.2	Callsigns and SSIDs on APRS-IS	4
3.3	Published server header parser	4
3.3.1	Callsign-SSID parser	4
3.3.2	Base TNC2 and AEA validation	4
3.3.3	AEA conversion	5
3.3.4	Third-party header validation	5
3.3.5	Client-header cleanup	5
4	Connecting a TCP client	5
4.1	Connection sequence	5
4.2	Login line	6
4.3	Originated packets	6
5	Ports and transports	6
5.1	Complete <code>javAPRSSrvr</code> port taxonomy	7
5.2	User-defined filtered feed	8
5.3	UDP delivery requested over TCP	8
5.4	HTTP, HTTPS, UDP, and WebSocket services	8
5.4.1	WebSocket stream	8
5.4.2	Send-only request format	8
5.4.3	HTTP authentication	9
5.4.4	Complete documented HTTP responses	9
6	q constructs	10
6.1	Defined q codes	10
6.2	Common examples	11
6.3	Complete q processing algorithm	11
6.3.1	Initial normalization for every packet	11
6.3.2	Packet entering through a direct no-login UDP input	11
6.3.3	Historical unverified input	12
6.3.4	Verified client-only input whose source differs from the login	12
6.3.5	Other accepted inbound connections	12
6.3.6	Packet from an outbound server connection	12
6.3.7	Common q validation, rejection, and tracing	12
6.4	Loop detection	12

7	Server processing	13
7.1	Duplicate detection	13
8	Server-side filtering	13
8.1	Filter forms	14
8.1.1	Geographic filters	14
8.1.2	Identity, path, object, and destination filters	14
8.1.3	Type filter	14
8.1.4	Symbol filter	15
8.1.5	q filter	15
8.1.6	Ways to set a filter	15
9	IGate operation	15
9.1	RF to APRS-IS	15
9.2	APRS-IS to RF message gating	16
9.3	Required third-party format on RF	16
9.4	Queries and capabilities	17
10	Interoperability and implementation checklist	17
10.1	Client checklist	17
10.2	Server checklist	17
10.3	IGate checklist	17
11	Limits and unresolved areas	17
12	Completeness matrix	18
13	Source register	19
14	Revision record	19

1 Status and scope

This document consolidates the complete APRS-IS specification subtree published on the APRS-IS web site, together with the operational policy on the site's front page and the implementation pages directly referenced by the specification. It is intended for authors of APRS clients, servers, and Internet Gateways (IGates), and for operators who need to distinguish APRS-IS behavior from RF APRS behavior.

It is an independent synthesis, not an official APRS-IS specification and not a replacement for the source pages maintained by Peter Loveall, AE5PL. When this document and a source page disagree, the source page takes precedence. The source register records the pages used and the date on which they were consulted.

This reference describes APRS-IS transport and gateway behavior. It does not reproduce the APRS application data formats defined by the APRS protocol specification, nor does it define AX.25 radio framing. Features documented only for `javAPRSSrvr` are identified as implementation-specific.

Completeness here means that every technical rule, option, algorithm branch, filter form, transport response, and example class from the source set is represented. It does not mean that copyrighted explanatory prose or Java source code is reproduced verbatim. The completeness matrix maps every source page to the corresponding sections of this reference.

1.1 Normative language

The official pages mix descriptive prose, recommendations, and explicit requirements. In this consolidation:

- **must** and **must not** identify requirements stated as mandatory by the source material;
- **should** and **should not** identify recommendations in the source material;
- **may** identifies an option;
- text labelled **javAPRSSrvr-specific** is not assumed to be a universal APRS-IS requirement.

The capitalization of these words has no independent RFC 2119 meaning. Their authority comes from the cited APRS-IS source, not from this compilation.

2 System model

APRS-IS is an Internet transport network for APRS packets. Clients establish low-level IP connections to servers and exchange APRS packets as delimited TNC2 monitor-format lines. Servers distribute accepted, non-duplicate packets to other interested clients and servers. [S1][S5]

The network began in the 1990s as Steve Dimse's mechanism for presenting RF APRS activity to browser users. The official overview describes an ad-hoc network organized around a central server core and records historical growth to more than 500 servers and 20,000 users. It also describes three or four core servers sharing the United States connection load. Those quantities are historical descriptive text, not fixed protocol requirements. [S1]

The network connects several different roles:

- an **originating Internet client** creates packets directly on APRS-IS;
- an **IGate** transfers eligible packets between RF and APRS-IS;
- an **APRS-IS server** validates connections, processes paths and q constructs, rejects loops and duplicates, and distributes packets;
- a **filtered client** subscribes to a subset of the distributed traffic;
- a **server peer** carries traffic between APRS-IS servers.

APRS-IS is a transport network, not a database and not a routing protocol that selects one RF gateway for each destination. A downstream database or web site can retain and display information that was not delivered more than once on a particular real-time server feed.

2.1 Network-use policy

APRS-IS exists to support amateur-radio APRS RF networks. Its published policy requires an Internet originator to assume that every packet can be gated to RF or can have arrived from RF. Packet contents and transmission rates must therefore be suitable for a busy shared radio channel. [S11]

The published maximum-rate guidance is:

Station or information class	Do not beacon faster than
Mobile station	1 minute
Weather station	5 minutes
Local infrastructure, such as a repeater	10 minutes
Fixed station	20 minutes

Non-amateur material must not be injected into APRS-IS while masquerading as an amateur station. It should use APRS objects where appropriate or a local-only network such as FireNet when it must remain outside APRS-IS. Software authors must not reuse another application's APRS destination identifier (**APxxxx**). They are responsible for distributing passcodes only to licensed amateur-radio operators and must not expose an unrestricted on-demand passcode service. [S11]

2.2 RF and Internet boundaries

The boundary between RF and APRS-IS is semantically important:

- q constructs exist only on APRS-IS and must never be transmitted on RF;
- TCPIP and TCPXX are Internet path markers and, except inside a correctly formed third-party packet, must not appear on RF;
- an Internet-to-RF packet uses APRS third-party format so that another IGate can recognize its Internet origin and avoid returning it to APRS-IS;
- an RF-to-Internet IGate normally preserves the received TNC2 line and adds only the appropriate q construct and IGate identity. [S2][S3][S6]

3 Line and packet representation

3.1 TNC2 monitor format

The normal APRS-IS representation is:

FROMCALL>TOCALL,PATH1,PATH2:information field

The first colon separates the header from the information field. A server must not modify the portion following that colon when forwarding an accepted packet. [S5]

TCP streams use CR/LF line termination. A complete line, including CR/LF, may not exceed 512 bytes. Lines beginning with # are server comments or control responses rather than APRS packets. [S2]

The older AEA monitor representation can be accepted by a server, but should be converted to TNC2 before redistribution. Non-standard or mangled headers must be repaired or rejected. [S5]

3.2 Callsigns and SSIDs on APRS-IS

For APRS-IS logins and textual packet headers, the published rules allow ASCII alphanumeric callsigns with an optional hyphen and one- or two-character alphanumeric SSID. The full callsign and SSID may not exceed nine characters. An explicit -0 is not used because an omitted SSID already represents zero. [S2][S5]

These Internet representation rules are wider than the AX.25 radio field. A station that will be addressed or transmitted on RF should therefore use an uppercase callsign and an AX.25-compatible numeric SSID from 0 through 15. Some radios will not respond to non-AX.25 SSIDs even when the packet is carried inside a third-party frame; the official page specifically identifies some Yaesu radios. [S2]

Case is not safely interchangeable:

- servers preserve the case of header fields;
- duplicate detection treats source and destination case as significant;
- passcode calculation uses the uppercase base callsign;
- equipment intended for RF interoperability should use uppercase callsigns and SSIDs. [S2][S5]

The connection page warns that future servers may restrict logins to the international amateur-callsign form. It publishes this regular expression:

```
(?>[1-9][A-Z][A-Z]?[0-9]|[A-Z][2-9A-Z]?[0-9])[A-Z]{1,4}
```

This prospective restriction is not the same as the wider textual identity syntax currently accepted by APRS-IS. [S2]

3.3 Published server header parser

The server-design page requires a server to clean or reject mangled TNC2/AEA headers and links to the current `javAPRSSrvr` parser as its concrete example. The following rules describe that implementation; they are included for completeness but must not be mistaken for an independently standardized wire grammar. [S5][S12]

3.3.1 Callsign-SSID parser

The implementation accepts a case-insensitive base form equivalent to:

```
[0-9A-Z]{1,9} ( "-" [0-9A-Z]{1,2} )?
```

The complete textual callsign-SSID is limited to nine characters. -0 is rejected by this parser rather than normalized, because normalization would affect duplicate comparison. This is stricter than the general server-design wording that says -0 is dropped; implementations must account for that published inconsistency. [S5][S12]

The implementation also provides a separate international amateur-callsign test. It accepts a 3-to-7-character base callsign shaped as one of #A#, #AA#, A#, A##, or AA#, followed by one to four letters, where # denotes the appropriate digit position. This test is distinct from the wider APRS-IS textual identity parser. [S12]

3.3.2 Base TNC2 and AEA validation

The published parser imposes these concrete limits:

- an origin field of 3 through 9 textual characters;
- a destination/path element of at most 9 textual characters;
- a non-empty information field of at most 256 bytes in this parser;
- a valid parsed origin with a base callsign at least 3 characters long;

- a valid destination whose base callsign is at least 2 characters long;
- only ASCII alphanumeric callsign and SSID components;
- no information field whose first byte is below 0x1c or equals 0x7f;
- no empty path component after normalization;
- at most one q construct or legacy I construct;
- a q construct must be followed by at least one callsign;
- a legacy I marker can appear only at the end and cannot be the first path element.

These parser limits coexist with the 512-byte APRS-IS line limit. The latter is a transport limit; it does not imply that this particular parser accepts a 256-byte information field plus arbitrary header length. [S2][S12]

An asterisk marks a used digipeater only before the q or legacy I portion of the path. If a malformed input places a used marker at or after that boundary, the implementation relocates the effective last-used-digipeater boundary to the last RF path component. [S12]

3.3.3 AEA conversion

An AEA line uses > separators differently from TNC2. The parser detects AEA form when the path contains >, separates its elements on > and comma, moves the destination into the TNC2 destination position, and retains the remaining elements as the path. It contains compatibility repairs for historical IGate strings in which a q or legacy I suffix was appended after the AEA destination. The redistributed result is TNC2, not AEA. [S1][S5][S12]

3.3.4 Third-party header validation

If the information field starts with }, the parser recursively validates the embedded TNC2 header. It rejects an immediately nested second }, a truncated embedded header, or an embedded representation that does not match the expected third-party path form. In the published Java implementation, the parsed embedded header must contain exactly two path entries and the second must be the effective last-used entry. The same callsign and header rules apply to the embedded header. [S12]

3.3.5 Client-header cleanup

The `javAPRSSrvr` client parser attempts to recover several monitor-mode artifacts before rejecting a line. Its cleanup logic:

- discards monitor text preceding the usable source header;
- removes an embedded <UI...> monitor annotation;
- removes a trailing port indicator beginning with /, a space, or <;
- removes trailing commas and > characters;
- trims surrounding whitespace;
- reconstructs a canonical `FROM>DEST,PATH:information` line and passes it through the base parser.

Cleanup never licenses a server to alter the information field. If a usable source, destination, or information field cannot be recovered within the published limits, the packet is rejected. [S5][S12]

4 Connecting a TCP client

4.1 Connection sequence

A TCP client performs the following sequence:

1. connect to a suitable server port;
2. receive a server identification comment;
3. send a login line;
4. receive a login response identifying the connection as verified or unverified;
5. exchange APRS packet lines and server comment lines.

The server identification has the conceptual form:

```
# software-name software-version
```

The login response has the conceptual form:

```
# logresp LOGIN VERIFIED-STATUS, server SERVERCALL [UDP]
[, ADJUNCT ["filter REQUESTED-FILTER"] [filter ACTIVE-FILTER active]]
```

The optional suffix reports UDP delivery and server-adjunct state, including the requested and active filters. Servers can periodically send other # comment lines as keepalives during an otherwise idle stream; clients ignore them as packet data. [S2][S5]

A bidirectional TCP client should disable the Nagle algorithm (TCP_NODELAY) to avoid unnecessary message latency. [S2][S5]

4.2 Login line

The login syntax is:

```
user MYCALL[-SSID] pass PASSCODE [vers SOFTWARE VERSION [UDP PORT] [COMMAND]]
```

SOFTWARE and VERSION contain no spaces. A passcode of -1 requests an unverified, receive-only connection. A client must be verified before it may inject packets into APRS-IS. The login callsign-SSID represents a station identity and is expected to be unique across APRS and AX.25. [S2]

Example of a receive-only filtered connection:

```
user NOCALL-10 pass -1 vers example 1.0 filter r/48.85/2.35/100
```

APRS-IS passcodes are an access-control convention intended to restrict injection to amateur-radio users. They are not cryptographic authentication, and a plain TCP connection does not provide confidentiality or integrity. javAPRSSrvr can accept a Base64-encoded login line in a connection stream; this is merely an alternate representation and provides no security. [S2]

The official connection page intentionally does not publish the passcode generation algorithm. It directs amateur-radio software authors to contact the maintainer and directs end users to obtain a passcode from their software author. This distribution policy is part of the documented access-control model even though the algorithm exists in open-source implementations. [S2]

4.3 Originated packets

A packet originated by the verified logged-in station uses only TCPIP* as its initial path:

```
MYCALL-10>APRS,TCPIP*:payload
```

The receiving server adds the appropriate q construct. A normal client must not invent a server-generated q construct. The only client-generated q constructs defined by the published documentation are those for IGates and special command or trace uses described later. [S2][S5][S6]

For a packet whose source is the verified login, the server design permits only TCPIP* before server q processing. A special qAZ or qAI construct can follow only with the originating station's own login as its initial identity. For ordinary client traffic, the connection page says that only IGate qAR and qAO are client generated; the q-construct page separately defines qAZ command and qAI trace packets as special client/server cases. [S2][S5][S6]

5 Ports and transports

The port numbers below are recommended defaults. An operator must consult the chosen server's status page because enabled services can differ. [S2]

	Port	Typical service	Notes
	14501	HTTP status	Can also expose other javAPRSSrvr HTTP features.
	10152	TCP full feed	Required on core servers; high traffic volume.
	14580	User-defined filtered feed	Includes message support plus the requested additive filter.
	23000	Full feed delivered by UDP	The TCP login must request a UDP destination port.

Port	Typical service	Notes
8080	HTTP or UDP send-only	Availability is server-dependent.

5.1 Complete `javAPRSSrvr` port taxonomy

The server implementation documentation defines the following port classes. Only the common numbers shown are conventions; a server status page is the authority for a particular host. [S13]

Port class	Direction	Adjunct/filter support	Behavior and common port
Full feed	Bidirectional	No	All packets except those received from that same client; commonly 23 or 10152.
Full feed with history	To client	No	Full feed preceded by the latest position for each station over an operator-selected history, commonly 30 minutes; commonly 10151.
Message-only	Bidirectional	Yes	Messages plus the sending station's positions; commonly 1314.
Client-only	Bidirectional	Yes	Messages addressed to the login, with associated positions; no standard port.
User-defined filter	Bidirectional	Yes	Client-only behavior, messages to stations gated by this client, and additive subscribed traffic; commonly 14580.
Local-only	Bidirectional	Yes	User-defined behavior, but client-injected traffic remains local to the server or subnet; no standard port.
Read-only	To client	Yes	Only traffic approved by the adjunct/filter; no standard port.
Filtered history	To client	Yes	Read-only filtered traffic preceded by matching historical positions; no standard port.
Multicast	To client	No	Multicast transmission of received packets; no standard port.
UDP send-only	To server	No	Either restricted local injection or a login plus packet; commonly 8080.
HTTP/WebSocket	HTTP is send-only; WebSocket bidirectional	WebSocket behaves as filtered	HTTP POST/PUT injection and WebSocket user-defined stream; commonly 8080.

The associated `javAPRSSrvr` admission algorithm has three states. With no login, no packets are accepted or delivered and the connection is closed after 30 seconds. A verified login can inject accepted packets. An unverified login is receive-only on APRS-IS. The implementation can optionally accept only the unverified client's own TCPIP* position and weather packets on a non-APRS-IS server; that exception is expressly unavailable on APRS-IS. [S5][S13]

5.2 User-defined filtered feed

Port 14580 starts with the traffic required for APRS messaging to the logged-in client and to stations gated by that client. A filter adds subscribed traffic; it does not replace the messaging traffic inherent to the port. Traffic sent from the client is not restricted by its receive filter. [S2][S8]

5.3 UDP delivery requested over TCP

A client can request that server-to-client packet delivery use UDP by adding `UDP port-number` to its TCP login. The TCP connection remains open for session state and keepalives. Each APRS line is placed in a separate UDP datagram without CR/LF. Packets from the client still travel over TCP; this mechanism is not the send-only UDP service. [S9]

Example sequence:

```
TCP client -> server: user NOCALL-10 pass -1 vers example 1.0 UDP 54321
TCP server -> client: # logresp NOCALL-10 unverified, server SERVER UDP
UDP server -> client: SRC>DEST,TCPIP*,qAC,ENTRY:payload
```

The server uses its own service port as the UDP source and the requested port as the client destination. The TCP connection remains authoritative for the session and receives keepalives. The official example shows a server comment on TCP every 20 seconds, but does not state that interval as a universal requirement. If the selected server port does not support UDP delivery, it ignores the request and continues sending packets over TCP. [S9]

5.4 HTTP, HTTPS, UDP, and WebSocket services

The official documentation describes several `javAPRSSrvr` services that are not universal APRS-IS requirements:

- HTTP or HTTPS POST/PUT can inject one or more packets;
- UDP can provide connectionless send-only injection;
- WebSocket can expose a stream similar to a TCP filtered port;
- Base64 login encoding only obscures a login and does not secure it;
- HTTPS or another protected transport is required when confidentiality is needed. [S2][S10]

The source records the implementation history: HTTP and UDP send-only ports appeared in `javAPRSSrvr` 3.15b01 in August 2009; APRS-IS HTTP authorization appeared in 4.3.2b76; HTTP Basic authentication in 4.3.3b40; and from 4.3.4 every configured HTTP/HTTPS port can provide status pages, send-only POST/PUT, and WebSocket service. These version markers describe `javAPRSSrvr`, not a negotiated APRS-IS protocol version. [S10]

5.4.1 WebSocket stream

On a supported `javAPRSSrvr` HTTP port, WebSocket accepts no protocol token or the tokens `readwrite` and `history`. The default is `readwrite`, equivalent to a user-defined filtered stream. `history` requires the filter in the login and prepends matching history. Server-to-client WebSocket messages are binary because APRS packets are not guaranteed to be UTF-8; client-to-server messages may be text or binary. The software name identifies the application or web site, and the version identifies the application version or page of origin. [S2]

5.4.2 Send-only request format

For HTTP send-only operation, current `javAPRSSrvr` documentation requires a single POST or PUT with, at minimum:

```
Accept-Type: text/plain
Content-Type: application/octet-stream
Content-Length: number-of-body-bytes
```

The body contains a login line followed by one or more TNC2 packet lines. If an HTTP authorization header supplies the login, a body login is optional and ignored. The body may use gzip content encoding. A packet can end in CR, LF, CR/LF, or no terminator when it is the final packet. [S10]

For UDP send-only operation, one datagram contains the login line followed by the TNC2 packet. UDP provides no receipt confirmation. A firewall-restricted local UDP insertion port can omit normal remote authentication and produces a qAU entry marker. [S10]

Send-only packets use qAO or qAo according to the q algorithm. A packet submitted with an invalid passcode is not accepted. The commonly documented service port is 8080; current examples also identify 8888 on `ametx.com` for secure HTTPS and WSS and `srvr.aprs-is.net:8080` for HTTP, WebSocket, and UDP testing, but availability is always operator-controlled. [S10]

The source illustrates rejection with this deliberately unverified request:

```
user TEST-1 pass -1 vers TestSoftware 1.0
TEST-1>APRS,TCPIP*:>This is a test packet
```

Because the login is unverified, the packet is not admitted to APRS-IS. [S10]

5.4.3 HTTP authentication

javAPRSSrvr supports three login carriers for HTTP-related services:

- the ordinary APRS-IS login line in the request body or connection stream;
- **Authorization: APRS-IS BASE64**, where BASE64 encodes the entire login line;
- HTTP Basic authentication, using `callsign-SSID` as the user name and the APRS-IS passcode as the password.

Base64 is not encryption. Basic authentication cannot carry a filter and is therefore principally intended for send-only HTTP. HTTPS is required when the credentials need transport confidentiality. [S2][S10]

5.4.4 Complete documented HTTP responses

Context	Code	Meaning or required accompanying information
Any HTTP request	405	Method is not HEAD, GET, POST, or PUT.
Status/static file	200	Valid status-page or configured-file response.
Status/static file	304	Configured file has not changed; the generated status page itself is not returned as 304.
Status/static file	404	Unknown page or file.
WebSocket	101	Protocol switched successfully; an authorization-info header can accompany authenticated setup.
WebSocket	401	Invalid authorization scheme or data; response includes WWW-Authenticate .
WebSocket	426	WebSocket upgrade failed.
POST/PUT	204	Valid login and packets accepted; authenticated responses include verified login information and X-Packetsrcvd reports the accepted count.

Context	Code	Meaning or required accompanying information
POST/PUT	400	No packet could be parsed or received.
POST/PUT	401	Invalid authorization scheme or data; response includes WWW-Authenticate .
POST/PUT	403	Invalid APRS-IS passcode.
POST/PUT	409	The user is already logged into this server.
POST/PUT	411	Missing Content-Length or value below 8.
POST/PUT	413	Content-Length exceeds 8192.
POST/PUT	415	Content type is not application/octet-stream .
POST/PUT	417	Declared content length differs from the received body length.

These HTTP details are implementation-specific and must not be assumed for a different APRS-IS server implementation. [S10]

6 q constructs

The q construct identifies how a packet entered APRS-IS and supports loop detection. It consists of a q code followed by one or more identifiers in the path. It is an APRS-IS mechanism and must not be transmitted on RF. [S6]

6.1 Defined q codes

- qAC (server generated)** Packet originated by the logged-in station on a verified bidirectional connection, with **FROMCALL** equal to the login. The following identity is the accepting server. This accompanies the existing **TCP/IP*** convention.
- qAX (server generated)** Historical marker for an unverified origin; deprecated and not propagated on current APRS-IS. It described a packet whose source equalled an unverified login and accompanied the historical **TCPXX*** convention.
- qAU (server generated)** Packet entered directly through a server UDP input. The following identity is the accepting server.
- qAo (server generated)** Gated packet received through a client-only connection where **FROMCALL** differs from the login and the input already identifies the logged-in IGate using legacy **,I**, **qAR**, or **qAr** semantics.
- qA0 (server generated or used in a client entry context)** Non-gated or send-only entry; also used by an RF IGate that cannot return messages to the heard station. On a client-only port it denotes an indirect packet that cannot be authenticated as a gated RF packet; on a send-only port it can identify a packet whose source matches the login.
- qAS (server generated)** Packet arrived from another server or was generated by a server without a usable q construct. It is also assigned to an indirect client packet that does not meet a more specific case. Server-generated APRS beacons are discouraged because APRS-IS is a virtual network.
- qAr (server generated)** RF-gated packet whose legacy **IGATE,I** entry was received indirectly. The following identity is the IGate, not necessarily the immediately connected client.
- qAR (IGate or server generated)** RF-gated packet uploaded by an IGate. An IGate can generate this directly; a server also obtains it by converting legacy **IGATE,I** when the identified IGate equals the verified direct login.

qAZ (client or server generated) Local command packet that must not be propagated. It normally carries connection-control traffic such as a message to USERLIST; the following identity is the generating server, client, or IGate.

qAI (client or server generated) Trace packet; server identities are accumulated after the construct. It starts with the originating identity and each traversed server can append its own identity under the algorithm below.

Capitalization is significant: **qA0**, **qAo**, **qAR**, and **qAr** do not mean the same thing. [S6]

A receive-only IGate uses **qA0** for all RF uploads. A bidirectional IGate also uses **qA0** for stations to which it will not return messages, including a station outside its configured local range or a translated source that is not itself APRS-capable. Receive-only IGates are discouraged on standard APRS frequencies when a carefully restricted bidirectional service is possible. [S6]

Legacy **IGATECALL,I** uploads can be converted by servers into **qAR**, **qAr**, or **qAo** according to the connection relationship. Client-generated q constructs were designed so a future authorization algorithm could validate them; the published pages do not define such a modern authorization mechanism. [S6]

An APRS-IS login must not be exactly eight characters drawn only from 0-9 and A-F, because that shape was reserved for a server-generated hexadecimal IP representation in historical q processing. [S6][S7]

6.2 Common examples

An Internet-originated packet submitted as:

```
NOCALL>APRS,TCPIP*:payload
```

can leave the accepting server as:

```
NOCALL>APRS,TCPIP*,qAC,SERVER:payload
```

An RF packet heard and uploaded by a bidirectional IGate is represented as:

```
NOCALL>APRS,WIDE1-1,qAR,IGATE:payload
```

If that IGate cannot send messages back to the station, it uses **qA0** rather than **qAR**. [S6]

With server tracing enabled, equivalent examples are:

```
NOCALL>APRS,TCPIP*,qAI,NOCALL,SERVER:payload
```

```
NOCALL>APRS,WIDE1-1,qAI,IGATE,SERVER:payload
```

Tracing increases network bandwidth and is intended for diagnosing suspected loops, not for routine indiscriminate use. [S6]

6.3 Complete q processing algorithm

The official algorithm page was last updated on 1 June 2012. The decision tree below preserves every published branch while expressing it independently of a particular programming language. **SERVER** is the processing server's login, **LOGIN** is the connected peer identity, and **IPHEX** is the deprecated eight-hex-character representation of a remote server address. [S7]

6.3.1 Initial normalization for every packet

1. Convert an accepted AEA representation to TNC2.
2. If a q code is the final path component and has no following identity, remove that incomplete q code.
3. If the packet has no q construct and its source equals the logged-in station, set its Internet path to **TCPIP*** for a verified connection or, historically, **TCPXX*** for an unverified connection.

6.3.2 Packet entering through a direct no-login UDP input

1. If a q construct has exactly one identity after it, replace the entire construct with **qAU,SERVER**.
2. If it has more than one identity, reject the header.
3. If no q construct exists, append **qAU,SERVER**.
4. Stop q processing for this branch.

6.3.3 Historical unverified input

This branch applies only when the source equals the unverified login and the packet has passed the historical conversion to TCPXX form. Packets failing that validation are dropped. Replace an existing `q` construct with `qAX,SERVER`, or append it when absent, and then stop `q` processing. Current APRS-IS server policy deprecates TCPXX and `qAX` and does not propagate unverified input. [S5][S7]

6.3.4 Verified client-only input whose source differs from the login

Apply the first matching transformation:

- `qAR,CALL` or `qAr,CALL` becomes `qAo,CALL`;
- `qAS,CALL` becomes `qAO,CALL`;
- `qAC,CALL` becomes `qAO,CALL` when `CALL` is neither this server nor the connected login;
- a final `CALL,I` becomes `qAo,CALL`;
- if none of these forms exists, append `qAO,LOGIN`.

Then proceed to the common `q`-validation and tracing phase.

6.3.5 Other accepted inbound connections

1. If a `q` construct already exists, leave its type unchanged and proceed to common `q` validation.
2. Otherwise, if the path ends in `CALL,I`, convert it to `qAR,CALL` when `CALL` equals the direct login, or to `qAr,CALL` otherwise.
3. Otherwise, if the source equals the login, append `qAO,SERVER` on a send-only UDP/HTTP port or `qAC,SERVER` on another port, then finish `q` processing.
4. Otherwise append `qAS,LOGIN` and proceed to common `q` validation.

6.3.6 Packet from an outbound server connection

When a packet arriving from a server peer lacks a `q` construct, convert a final `CALL,I` to `qAr,CALL`. If it has neither form, the historical algorithm appends `qAS,IPHEX`; that hexadecimal-address behavior is deprecated. [S7]

6.3.7 Common `q` validation, rejection, and tracing

Apply these tests in order:

1. Reject and log a packet containing `qAZ`; it is local control traffic.
2. Reject and log `qAC` unless the complete path preceding it is exactly `TCP/IP*`.
3. If `SERVER` occurs anywhere after the `q` code, log a loop with the sender's IP address and stop processing.
4. If any `callsign-SSID` occurs twice after the `q` code, log a loop and stop.
5. If another verified login occurs after the `q` code and that identity is not explicitly allowed multiple verified connections, log a loop and stop. The historical `IPHEX` identity of an outbound connection counts as a verified login for this test.
6. For an inbound client, if its `LOGIN` occurs after the `q` code but is not the final path identity, log a loop and stop.
7. If server tracing is enabled, the `q` type is already `qAI`, or the source is on the trace list, extend the trace: append `LOGIN` for a verified inbound port when it is not already present; append `IPHEX` for an outbound server connection; then append `SERVER`.

The algorithm's rejection logs are diagnostic implementation behavior. The observable protocol requirement is that identified loops and `qAZ` local traffic are not propagated. [S6][S7]

6.4 Loop detection

The `q` algorithm rejects several loop indicators, including:

- the receiving server's own login appearing after the `q` construct;
- the same `callsign-SSID` occurring twice after the `q` construct;
- a conflicting verified login in the trace;
- an inbound client's login appearing in an invalid position in the trace.

Servers and packet-injecting clients must use distinct logins. Reusing one identity for a server and another local injector can make valid traffic appear to be a loop. Servers should have only one upstream server connection and must not echo traffic to the connection from which it arrived. [S5][S6][S7]

7 Server processing

For an accepted packet, a conforming server performs at least these operations:

1. validate the connection and packet header;
2. normalize an accepted AEA line to TNC2 if necessary;
3. apply the q algorithm;
4. detect loops;
5. perform duplicate detection;
6. distribute each non-duplicate packet without changing its information field to the clients entitled to receive it. [S5][S7]

7.1 Duplicate detection

The published server design defines a duplicate key using:

- the origin callsign and SSID;
- the destination callsign, ignoring its SSID;
- the information-field length;
- the information-field content.

The RF or APRS-IS path is deliberately ignored. Source and destination case is significant. Duplicate checking uses a 30-second sliding window for each packet. [S5]

Consequences include:

- copies received through different IGates can be duplicates even though their paths differ;
- a content change, including a meaningful length change, can prevent a match;
- copies separated by more than the window can both be distributed;
- a non-conforming component that strips or changes whitespace or control characters can interfere with duplicate recognition;
- server duplicate suppression is not an election mechanism between IGates.

Local duplicate handling at an IGate is a separate implementation function, not part of the documented APRS-IS server duplicate algorithm. It can be useful for RF digipeating, transmit-loop prevention, or device-specific processing, but it must not be described as the APRS-IS server mechanism.

8 Server-side filtering

A receive filter is an additive subscription applied to traffic flowing from the server to the client. It does not reject packets sent from that client into APRS-IS. Exclusion terms remove matches from the added subscription but do not disable the port's mandatory messaging behavior. [S8]

Filtered-port messaging behavior includes messages addressed to the logged-in client and to stations gated by that client. The server must also deliver the next available position packet from a station that sends one of those messages. [S2][S5][S8]

A filter can be supplied in the login line, in a server comment command, or in an APRS message to **SERVER** where supported. The preferred form is the login line:

```
user NOCALL pass -1 vers example 1.0 filter r/48.85/2.35/100 t/m
```

The source gives these additional combination examples:

```
filter r/33/-97/200 t/n
filter m/200 -p/CW
```

The first adds traffic within 200 km of Dallas and NWS traffic. The second adds traffic within 200 km of the logged-in station while excluding source prefix **CW**. [S8]

8.1 Filter forms

Form	Purpose
r/lat/lon/km	Positions and objects in a radius.
m/km	Radius around the last known position of the logged-in client.
f/call/km	Radius around the last known position of another station or object.
a/north/west/south/east	Geographic bounding box.
p/prefix/...	Source callsign prefixes.
b/call/...	Exact source callsigns, with supported prefix wildcard.
o/name/...	Object or item names.
os/name/...	Strict object-name matching.
t/types	APRS packet-type categories.
s/primary/alternate/overlay	Symbol table and overlay matching.
d/digi/...	A digipeater appearing in the packet path.
e/entry/...	Entry identity immediately following the q construct.
g/addressee/...	APRS message addressee or group.
u/destination/...	TNC2 destination or to-call.
q/codes/I	q-construct categories and optional IGate positions.

Multiple positive terms are combined as alternatives: a packet matching any included term is selected. Prefixing a term with `-` excludes its matches from the subscribed traffic. `filter default` restores the predefined filter for the port. Coordinates use signed decimal degrees, with south and west negative. [S8]

8.1.1 Geographic filters

- `r/lat/lon/km` selects positions and objects inside the radius and messages addressed to stations in that radius. Up to nine range filters can coexist.
- `a/north/west/south/east` selects a rectangular area expressed as its north-west and south-east bounds. Up to nine area filters can coexist.
- `m/km` is a range centered on the last known position of the logged-in client.
- `f/call/km` is a range centered on the last known position of a named station or object. Up to nine friend filters can coexist.

8.1.2 Identity, path, object, and destination filters

- `p/AA/BB/...` selects packets whose source begins with any listed prefix.
- `b/CALL1/CALL2/...` selects exact source identities; `*` is permitted only as a suffix wildcard denoting a prefix.
- `d/DIGI1/DIGI2/...` selects packets containing a listed digipeater identity in the path and supports the same prefix wildcard.
- `e/ENTRY1/ENTRY2/...` selects the identity immediately following the q construct, allowing selection by receiving IGate; it supports the prefix wildcard.
- `u/DEST1/DEST2/...` selects the TNC2 destination/to-call and supports the prefix wildcard.
- `g/ADDRESSEE1/ADDRESSEE2/...` selects APRS messages addressed to a listed station or group and supports the prefix wildcard.
- `o/NAME1/NAME2/...` selects exact object or item names. Within a filter term, `|` represents the `/` character and `~` represents `*` so those characters do not conflict with filter syntax. APRS objects have nine-character names; items have names from three through nine characters.
- `os/NAME1/NAME2/...` provides strict object-name matching with the same substitutions and wildcard support. Only one strict-object filter is allowed, and it must be the final filter term.

8.1.3 Type filter

`t/types` selects one or more APRS categories. The defined letters are:

Letter	Category
p	position
o	object
i	item
m	message
q	query
s	status
t	telemetry
u	user-defined
n	NWS-format messages and objects
w	weather, including the positions associated with positionless weather reports

The extended form `t/types/call/km` limits the selected types to a radius around the last known position of `call`. [S8]

8.1.4 Symbol filter

The form is `s/primary/alternate/overlay`. `primary` lists symbols from the primary table; `alternate` lists symbols from the alternate table; and `overlay` restricts alternate symbols to the listed case-sensitive overlays. Within the symbol lists, `|` represents `/`. Published examples include:

```
s/->      primary-table House and Car
s//#      alternate-table Digi, with or without overlay
s//#/T    alternate-table Digi with uppercase T overlay
```

8.1.5 q filter

`q/codes/I` selects case-sensitive q-code suffixes. For example, `q/C` selects `qAC`, while `q/rR` selects `qAr` and `qAR`. The optional `I` flag selects position packets from IGates that have been identified elsewhere by `qAr`, `qAo`, or `qAR`; `q/I` requests that set without specifying a q-code list. [S8]

8.1.6 Ways to set a filter

A writable connection can set a filter in three ways:

```
user NOCALL pass -1 vers example 1.0 filter r/48.85/2.35/100
#filter r/48.85/2.35/100
NOCALL>APRS::SERVER :filter r/48.85/2.35/100
```

The login form is preferred. The comment form changes it during an existing stream. In the APRS-message form, the server acknowledges a sequenced request and returns the result; the addressee normally must be uppercase or literally `SERVER` for clients that recognize only uppercase server identities. Read-only and filtered-history ports can use only the login form because the client cannot transmit commands on those ports. [S8]

9 IGate operation

An IGate has two distinct directions with different rules:

- **RF to APRS-IS:** upload valid eligible RF packets;
- **APRS-IS to RF:** selectively transmit messages and related information to stations considered local.

The second direction is intentionally restrictive because APRS-IS has far more capacity than a shared RF channel. [S3]

9.1 RF to APRS-IS

For AX.25 RF, the source rules require a valid CRC, UI control field `0x03`, and no-layer-3 PID `0xF0`. A TNC in `PASSALL` mode cannot provide the required validity assurance. [S3][S4]

An IGate uploads eligible RF packets except for the following categories:

- a packet that fails the applicable link-layer validity checks;
- a third-party packet whose inner header contains TCPIP or TCPXX;
- a generic APRS query;
- a packet whose header contains TCPIP or TCPXX;
- optionally, a packet carrying NOGATE or RFONLY. [S4]

For a third-party RF packet that does not contain the Internet-origin markers, the IGate removes the outer RF header and the } third-party indicator before uploading the inner packet. The higher-level IGate design page additionally states that path information in the third-party header is stripped to reduce bandwidth and remain within AX.25 length constraints. Implementations must preserve the inner information field while applying this documented third-party exception. [S3][S4]

The IGate otherwise preserves the packet and appends:

,qAR,IGATECALL

for a messaging-capable IGate, or:

,qAO,IGATECALL

when it cannot provide message return service to that station. It must not alter the information field. [S3][S4][S6]

9.2 APRS-IS to RF message gating

The published criteria permit a message and associated position to be gated to RF when all of the following are true:

1. the recipient has been heard locally within a configured recent period;
2. the sender has not recently been heard on RF, excluding packets that were themselves gated from the Internet;
3. the sender's header does not contain TCPXX, NOGATE, or RFONLY;
4. the recipient has not recently been identified as an Internet station or IGate. [S4]

For this test, a station has been heard through the Internet when its packets contain TCPIP* or TCPXX*, or when RF carries a third-party packet gated by that station whose inner header contains TCPIP or TCPXX. The latter case identifies the station as an IGate even though the observation itself occurred on RF. [S4]

The design guidance recommends considering a station local for no more than one hour and limiting both the transmit path and the definition of local to the minimum coverage needed. Digipeater-hop information is preferred over a simple geographic distance when practical. [S3]

After gating a message, the IGate should send the next position packet from the sending station rather than injecting an old stored position. This gives the RF recipient context without placing historical position data on RF. [S3]

The source criteria define eligibility but do not define an election protocol among multiple overlapping bidirectional IGates. Therefore several eligible IGates can independently transmit the same APRS-IS message. APRS-IS duplicate suppression upstream does not by itself coordinate those RF transmissions.

Separately from automatic message gating, the published criteria allow the operator to configure explicit RF gating by callsign, object name, or another local policy. This is discretionary traffic and remains subject to the RF capacity warning: an IGate has access to an Internet stream far exceeding the shared radio channel and must not gate it indiscriminately. [S3][S4]

9.3 Required third-party format on RF

An Internet packet gated to RF uses this structure:

IGATECALL>APRS,GATEPATH:}FROMCALL>TOCALL,TCPIP,IGATECALL*:original information

The APRS-IS path is removed before transmission. The inner TCPIP,IGATECALL* path marks the packet as Internet-originated so another IGate will not upload it again. Neither a q construct nor the obsolete I construct is transmitted on RF. [S4]

GATEPATH is selected by the IGate operator or implementation. The official criteria do not specify how to discover a usable reverse RF route to a station that was heard through an asymmetric digipeater path.

9.4 Queries and capabilities

An IGate does not pass generic queries between RF and APRS-IS because one broadcast query could cause a large number of responses. It can answer a directed ?IGATE? query with an APRS station-capabilities packet beginning with:

```
<IGATE,MSG_CNT=n,LOC_CNT=n
```

Additional operator information may follow. [S3]

10 Interoperability and implementation checklist

10.1 Client checklist

- Select the correct server and port rather than assuming every server exposes the same services.
- Use TCP_NODELAY on a bidirectional TCP stream.
- send a syntactically valid login and identify the software and version;
- treat # lines as server comments or responses;
- preserve packet octets that the application does not explicitly interpret;
- do not transmit with an unverified connection;
- originate packets with TCPIP* and do not copy observed server q constructs;
- reconnect with backoff rather than creating a tight reconnect loop.

10.2 Server checklist

- identify the server on connection and return an explicit login result;
- validate logins and prohibit injection by unverified clients;
- accept and distribute TNC2 lines with CR/LF framing;
- preserve the information field;
- apply the q algorithm and loop protection;
- apply the documented 30-second duplicate window;
- maintain a single upstream path and never echo to the source connection;
- preserve APRS messaging behavior when offering filtered ports.

10.3 IGate checklist

- keep RF-to-IS and IS-to-RF policy separate;
- upload all eligible valid RF packets, not only packets understood as APRS;
- reject Internet-originated third-party packets from RF-to-IS gating;
- preserve RF packet data and add the appropriate qAR or qAO entry marker;
- gate only eligible messages and related positions to local RF stations;
- use third-party format for Internet-to-RF traffic;
- never place q constructs on RF;
- limit the RF path and traffic volume;
- do not treat APRS-IS deduplication as coordination between IGates.

11 Limits and unresolved areas

This section is editorial analysis derived from the consolidated requirements; it is not normative APRS-IS source text. The source material leaves several operational questions outside the APRS-IS protocol definition:

- There is no distributed election procedure selecting one of several overlapping IGates for an Internet-to-RF message.
- The official criteria do not define a learned reverse path for asymmetric RF digipeater networks.
- Terms such as “within range,” “predefined time period,” and “local” require operator or implementation policy.
- The APRS-IS passcode does not provide modern authentication security.

- Several transport features are specified as `javAPRSSrvr` behavior rather than as implementation-independent protocol requirements.
- The published q algorithm includes historical and deprecated paths that new implementations must recognize carefully without generating obsolete forms.
- A downstream archive can preserve multiple observations even when a real-time APRS-IS server suppresses duplicate distribution.

These gaps must be identified as such. They must not be silently filled by promoting one implementation's behavior to a network-wide requirement.

12 Completeness matrix

This matrix defines the source boundary for the complete-source draft. The APRS-IS web site's software catalogue, public services, D-PRS tools, and live activity displays are products or services rather than APRS-IS transport specifications and are intentionally outside this document.

Official or directly referenced source	Complete technical coverage in this document
APRS-IS front page [S11]	Network purpose, RF-suitable traffic policy, rate guidance, objects, destination identifiers, and passcode distribution.
APRS-IS Specifications [S1]	Architecture, TNC2/AEA transport, core network, q processing, and filtering context.
Connecting to APRS-IS [S2]	TCP setup, TCP_NODELAY, ports, WebSocket, login and response forms, line limits, comments, originated paths, and identity rules.
Client UDP [S9]	TCP-maintained session, UDP packet framing, source/destination port behavior, fallback, and example exchange.
Send-only Ports and Formats [S10]	HTTP/HTTPS/UDP, headers, authentication, request body and termination, gzip, q behavior, response codes, and local UDP insertion.
Server-side Filter Commands [S8]	Additive and negative semantics, every filter form, category letters, limits, substitutions, examples, reset, and all command channels.
IGate Design [S3]	Both IGate directions, link validity, third-party handling, queries, associated positions, locality, validated login, preservation, and RF-capacity policy.
IGate Details [S4]	Complete RF-to-IS exclusions, IS-to-RF eligibility tests, Internet-heard definition, discretionary gating, q markers, and RF third-party path.
Server Design [S5]	Identification and login response, validation, TNC2 preservation, AEA normalization, identity limits, q requirement, duplicates, topology, case, deprecated unverified input, and filtered messaging.
q Construct [S6]	Every server/client q code and case distinction, legacy conversion, identity uniqueness, tracing, examples, and reserved hexadecimal-login shape.
q Algorithm [S7]	Every normalization, input-class transformation, rejection, loop-detection, and trace-extension branch.
Server Header Parsing [S12]	Callsign parsing, base limits, TNC2/AEA conversion, path/q/I validation, third-party validation, and client-header cleanup.
Common Client Port Types [S13]	Every documented <code>javAPRSSrvr</code> port class and its login admission states.

13 Source register

All web sources were consulted on 28 September 2026.

[S1] **APRS-IS Specifications.** Overview and history of the network, TNC2 transport model, and relationship to filtering and q constructs.

<https://www.aprs-is.net/Specification.aspx>

[S2] **Connecting to APRS-IS.** TCP behavior, ports, login syntax, line size, originated packet paths, callsign rules, and WebSocket notes.

<https://www.aprs-is.net/Connecting.aspx>

[S3] **IGate Design.** IGate purpose, RF validity requirements, locality, queries, message-associated positions, and preservation rules.

<https://www.aprs-is.net/IGating.aspx>

[S4] **IGate Details.** RF-to-IS exclusions, IS-to-RF message criteria, q entry markers, and mandatory third-party RF format.

<https://www.aprs-is.net/IGateDetails.aspx>

[S5] **Server Design.** Required server behavior, TNC2 distribution, duplicate key and window, upstream topology, and messaging behavior on filtered ports.

<https://www.aprs-is.net/ServerDesign.aspx>

[S6] **q Construct.** Meanings of the q codes, client and server generation, examples, case distinctions, and identity constraints.

<https://www.aprs-is.net/q.aspx>

[S7] **q Algorithm.** Processing order and detailed loop-detection algorithm; the page reports that the algorithm was last updated on 1 June 2012.

<https://www.aprs-is.net/qalgorithm.aspx>

[S8] **Server-side Filter Commands.** Additive-filter semantics, command delivery, exclusion behavior, and available filter forms.

<https://www.aprs-is.net/javAPRSFilter.aspx>

[S9] **Client UDP.** TCP-maintained sessions with server-to-client UDP packet delivery.

<https://www.aprs-is.net/ClientUDP.aspx>

[S10] **Send-only Ports and Formats.** `javAPRSSrvr` HTTP, HTTPS, UDP, and authorization behavior for send-only clients.

<https://www.aprs-is.net/SendOnlyPorts.aspx>

[S11] **APRS-IS.** Network purpose and operating policy for Internet-originated traffic, beacon rates, objects, APRS destination identifiers, and passcodes.

<https://www.aprs-is.net/Default.aspx>

[S12] **Server Header Parsing.** Published `javAPRSSrvr` parsing example for callsign-SSID validation, TNC2/AEA conversion, third-party parsing, and cleanup of mangled client headers.

<https://www.aprs-is.net/Server%20Header%20Parsing.aspx>

[S13] **Common Client Port Types.** Complete `javAPRSSrvr` port taxonomy and login-state admission behavior.

<https://www.aprs-is.net/javAPRSSrvr/ports.aspx>

14 Revision record

Date	Revision	Description
2026-09-28	0.1 draft	Initial English consolidation of the published APRS-IS specification pages.

Date	Revision	Description
2026-09-28	0.2 complete-source draft	Added every rule and algorithm branch from the official specification subtree, front-page policy, linked header parser, and complete server port taxonomy.